Research Report and Findings

1. Technical Implementation: Two-Agent Architecture

Our implementation leverages OpenAI's capabilities through a two-agent system designed to combine visual verification with web-based fact-checking.

Agent 1: User Agent (Browser Controller)

Purpose: Visual data collection and interaction with the Arcade platform.

Responsibilities:

- Authenticates to the Arcade platform as a registered user
- Navigates to individual game submission pages
- Captures screenshots of games as they render on the platform
- Extracts visible metadata (title, description, developer name, category)
- Passes collected data to the Web Search Agent

Technical Stack:

- Browser automation framework (Playwright/Selenium/Puppeteer)
- Screenshot capture functionality
- Session management for authenticated browsing

Agent 2: Web Search Agent (Verification & Analysis)

Purpose: External verification and recommendation generation.

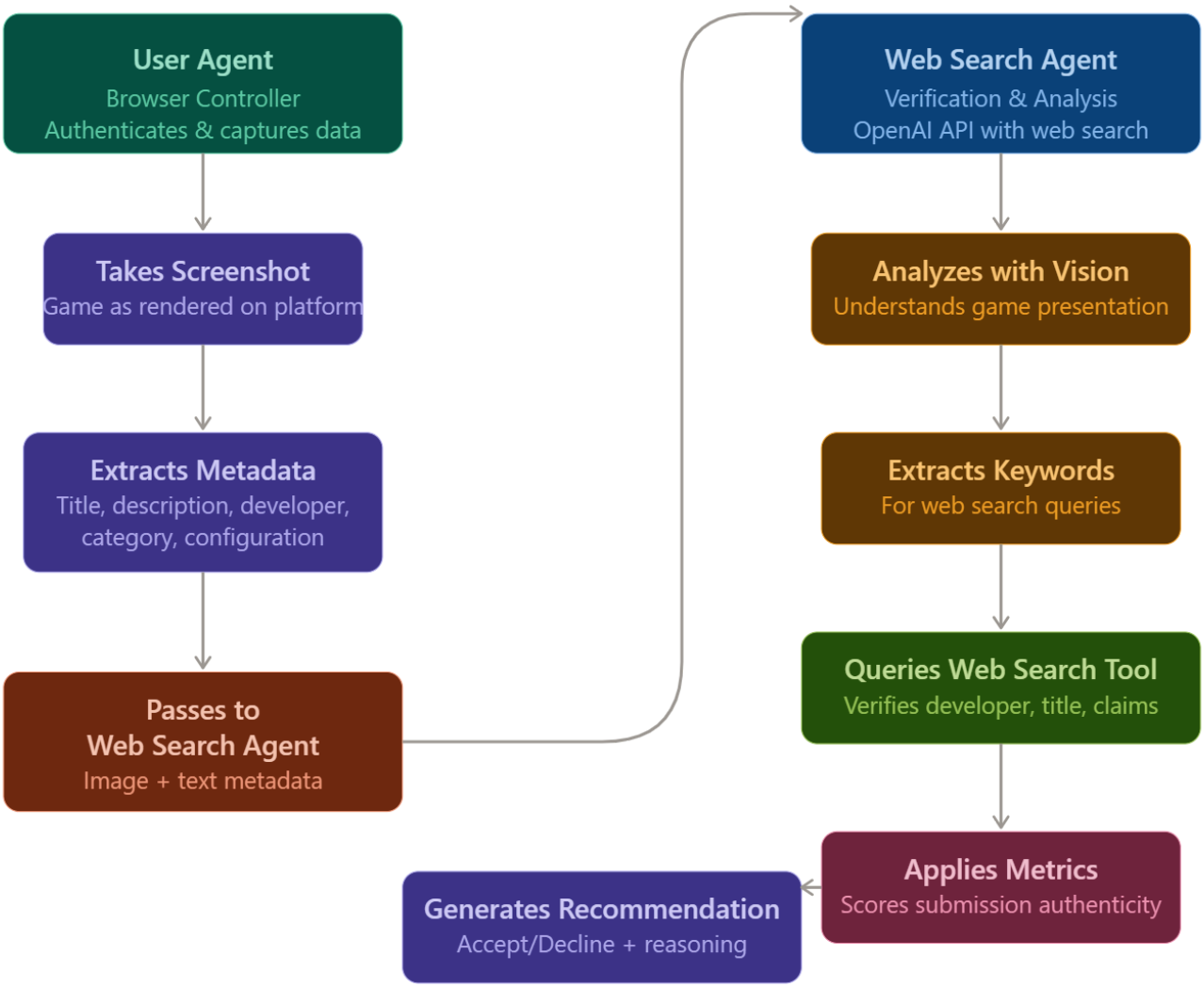
Responsibilities:

- Receives game screenshots, title, and description from User Agent
- Uses OpenAI's web search tool to verify submitted information
- Cross-references developer names, game titles, and claimed attributes
- Applies evaluation metrics to score submission authenticity
- Generates recommendation with supporting reasoning

Technical Stack:

- OpenAI Responses API with web search tool enabled (Web search | OpenAI API)
- Multimodal input processing (text + images)
- Structured output for evaluation metrics

How the Agents Collaborate



Key Integration Point: The User Agent passes both the screenshot (as image input) and extracted text metadata to the Web Search Agent. The Web Search Agent's vision model analyzes the screenshot to understand the visual presentation, then uses that understanding along with the text metadata to formulate web search queries for verification.

2. Data Accuracy & Verification

To ensure submitted game information is trustworthy, the AI goes beyond reading metadata in isolation. It actively uses search and fetch tools to cross-reference key fields against publicly available data, and applies evaluation metrics to score each submission before producing a recommendation.

How Verification Works

Search Tools: The AI queries external sources using OpenAI's web search tool to confirm whether a developer, studio, or game title exists and matches the submitted description. The tool accepts text queries generated from both the extracted metadata and visual analysis of the game's presentation.

Fetch Tools: When the web search tool returns relevant URLs, the AI can retrieve full web pages or data sources to validate specific claims in the game's description, such as genre, platform, or publisher details.

Visual Analysis: Screenshots captured by the User Agent are analyzed to verify consistency between what the game claims to be (in metadata) and what it visually presents. This helps detect mismatched categories or misleading descriptions.

Evaluation Metrics: Each submission is scored across multiple dimensions before a recommendation is produced. If data cannot be verified or scores fall below threshold, the AI flags the inconsistency in its reasoning and adjusts its recommendation accordingly.

Evaluation Metrics

Metric	What It Checks
Title Authenticity	Does the game title match known records or appear fabricated?
Developer Credibility	Is the developer name a real or recognizable entity?
Description Quality	Is the description sufficiently detailed and consistent with the category?
Category Accuracy	Does the described content align with the claimed genre/category?
Data Consistency	Are all submitted fields internally consistent with each other?
Visual-Metadata Alignment	Does the screenshot match the claimed game type and description?

3. Implementation Considerations

API Usage

Since we are using OpenAI's API key, all agent calls will route through OpenAI's Responses API with the web search tool enabled. The multimodal capability allows us to pass screenshots alongside text in a single API request.

Error Handling

- If the User Agent fails to capture a screenshot, the Web Search Agent proceeds with text-only verification
- If web search returns no results for a claimed game/developer, this is flagged as a red flag in the evaluation metrics
- Network timeouts and API rate limits are handled with exponential backoff

Privacy & Security

- The User Agent operates with dedicated credentials separate from admin accounts
- Screenshots are processed in-memory and not persistently stored
- All verification queries are logged for audit purposes

Performance Optimization

- User Agent operates on a queue system to process submissions sequentially
- Web Search Agent batches metadata extraction to minimize API calls
- Caching mechanism for frequently verified developers/studios